

Problem 1: FSM design

1. State Diagram

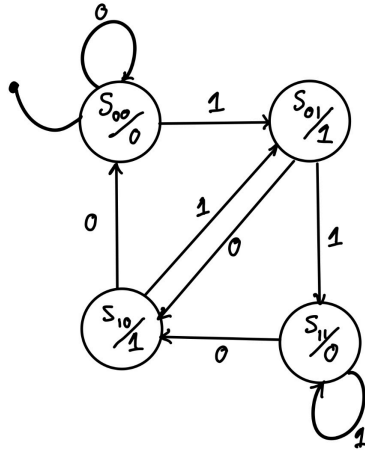


Figure 1: Moore FSM diagram

2. Transition Table and State Encodings

Table 1: Transition Table

State	Input: 0	Input: 1	Output
S00	S00	S01	0
S01	S10	S11	1
S10	S00	S01	1
S11	S10	S11	0

Table 2: State Encodings

State	S1	S0
S00	0	0
S01	0	1
S10	1	0
S11	1	1

3. Next State, Output Logic and Circuit Schematic

Table 3: Next State Table

Input	S1	S0	S1+	S0+
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	1	0
1	0	0	0	1
1	0	1	1	1
1	1	0	0	1
1	1	1	1	1

Table 4: Output Table

S1	S0	Out
0	0	0
0	1	1
1	0	1
1	1	0

$$\begin{aligned}
 S_1^+ &= \overline{1n} \overline{S_1} S_0 + \overline{1n} S_1 \overline{S_0} + 1n \overline{S_1} S_0 + 1n S_1 \overline{S_0} \\
 &= \overline{1n} S_0 (\overline{S_1} + S_1) + 1n S_0 (\overline{S_1} + S_1) \\
 &= \overline{1n} S_0 + 1n S_0 \\
 &= S_0
 \end{aligned}$$

$$\begin{aligned}
 S_0^+ &= 1n \overline{S_1} \overline{S_0} + 1n \overline{S_1} S_0 + 1n S_1 \overline{S_0} + 1n S_1 S_0 \\
 &= 1n \overline{S_1} (\overline{S_0} + S_0) + 1n S_1 (\overline{S_0} + S_0) \\
 &= 1n (\overline{S_1} + S_1) \\
 &= 1n
 \end{aligned}$$

$$Out = \overline{S_1} S_0 + S_1 \overline{S_0}$$

$$= \overline{(\overline{S_1} S_0)(S_1 \overline{S_0})}$$

Figure 2: Logic Simplification steps for Next State and Output

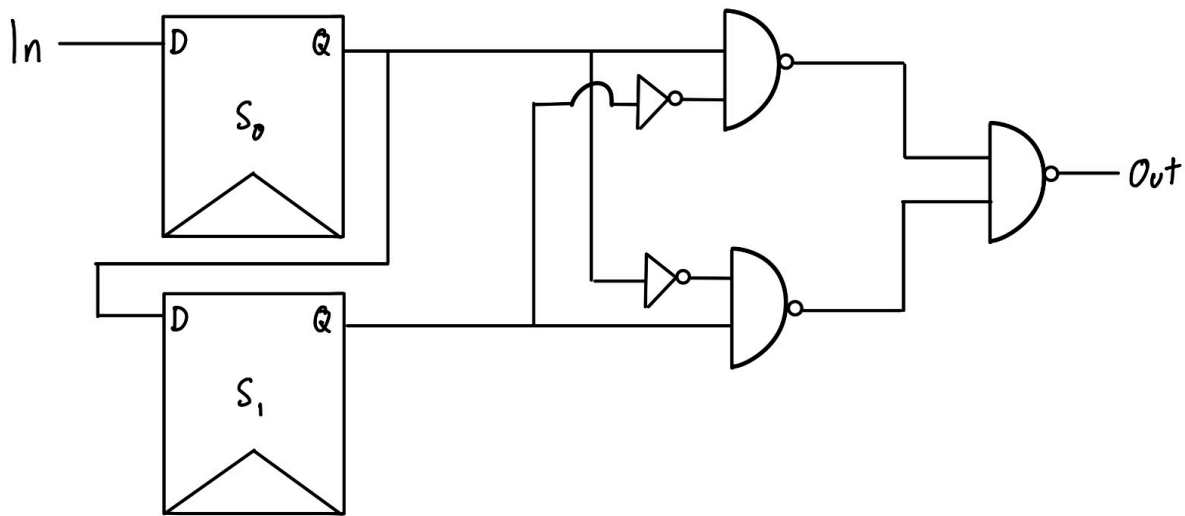


Figure 3: Circuit Diagram of FSM

4. RTL code and Xcelium Testing

RTL code for the FSM along with the testbench can be found in the Appendix

Xcelium Testing

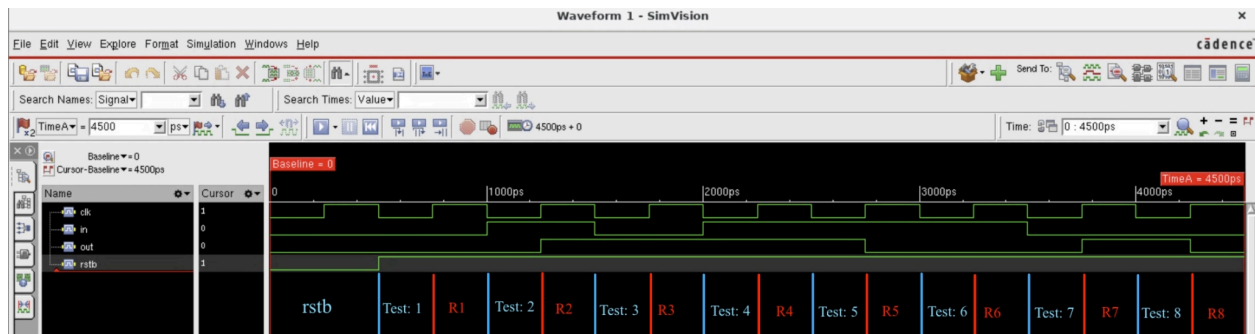


Figure 4: Results of Xcelium simulation with labelled Test Cases (inputs on clock negative edge) and Results (outputs on clock positive edge)

In this diagram, on each blue line, the inputs are put into the FSM on the falling edge of the clock and the results can be observed on the red line (rising edge of clock). Each test is named test #, directly followed by R# corresponding to each test. Expected and actual results are shown in Table 5.

Table 5: Validation Table for Problem 1 FSM

Test Case	Input	Expected Transition	Expected State	Expected Output	Actual Output (R#)
rstb	rstb = 0	–	S00	0	0
1	0	S00 → S00	S00	0	0
2	1	S00 → S01	S01	1	1
3	0	S01 → S10	S10	1	1
4	1	S10 → S01	S01	1	1
5	1	S01 → S11	S11	0	0
6	1	S11 → S11	S11	0	0
7	0	S11 → S10	S10	1	1
8	0	S10 → S00	S00	0	0

Note: Inputs for test cases always on falling edge of clock, and results are shown on the rising edge of clock, ensuring no race conditions

5. Synthesis

SDC file:

```
create_clock -name clk -period 0.5 -waveform { 0 0.25 } [get_ports clk]
```

```
# ----- Input constraints -----
```

```
set_input_delay -clock clk -max 0.2 [get_ports {rstb in}]
```

```
set_input_delay -clock clk -min -0.2 [get_ports {rstb in}]
```

```
# ----- Output constraints -----
```

```
set_output_delay -clock clk -max 0.2 [get_ports out]
```

```
set_output_delay -clock clk -min -0.2 [get_ports out]
```

```
set_max_delay 0.4 -from [all_inputs] -to [all_outputs]
```

```
# Assume 50fF load capacitances everywhere:
```

```
set_load 0.050 [all_outputs]
```

```
# Set 10fF maximum capacitance on all inputs
set_max_capacitance 0.010 [all_inputs]
```

```
# set clock uncertainty of the system clock (skew and jitter)
#set_clock_uncertainty -setup 0.03 [get_clocks clk*]
#set_clock_uncertainty -hold 0.06 [get_clocks clk*]
```

```
# set maximum transition at output ports
set_max_transition 0.07 [current_design]
```

```
# set_attr use_scan_seqs_for_non_dft false
```

Table 6: Screenshots of area, cell counts and timing reports from synthesis

Report type	Screenshot
Area/Cell Counts	<pre> Instance Module Cell Count Cell Area Net Area Total Area ----- transition_FSM 1 5.320 1.620 6.940 </pre>
Timing	<pre> Path 1: MET (204 ps) Late External Delay Assertion at pin out Group: clk Startpoint: (R) Sreg_reg[0]/CK Clock: (R) clk Endpoint: (R) out Clock: (R) clk Capture Launch Clock Edge:+ 500 0 Src Latency:+ 0 0 Net Latency:+ 0 (I) 0 (I) Arrival:= 500 0 Output Delay:- 200 Required Time:= 300 Launch Clock:- 0 Data Path:- 96 Slack:= 204 Exceptions/Constraints: output_delay 200 transition_FSM.sdc_line_12 #----- # Timing Point Flags Arc Edge Cell Fanout Load Trans Delay Arrival Instance # (fF) (ps) (ps) (ps) Location #----- Sreg_reg[0]/CK - - R (arrival) 1 - 0 - 0 (-,-) Sreg_reg[0]/Q - - CK->Q R DFFR_X1 1 0.4 7 96 96 (-,-) out <<< - R (port) - - - 0 96 (-,-) #----- </pre>

Problem 2: Memory

1-3. Writing RTL and TB code

All RTL and TB code can be found in the appendix

4. Xcelium Simulation Results

Xcelium Testing

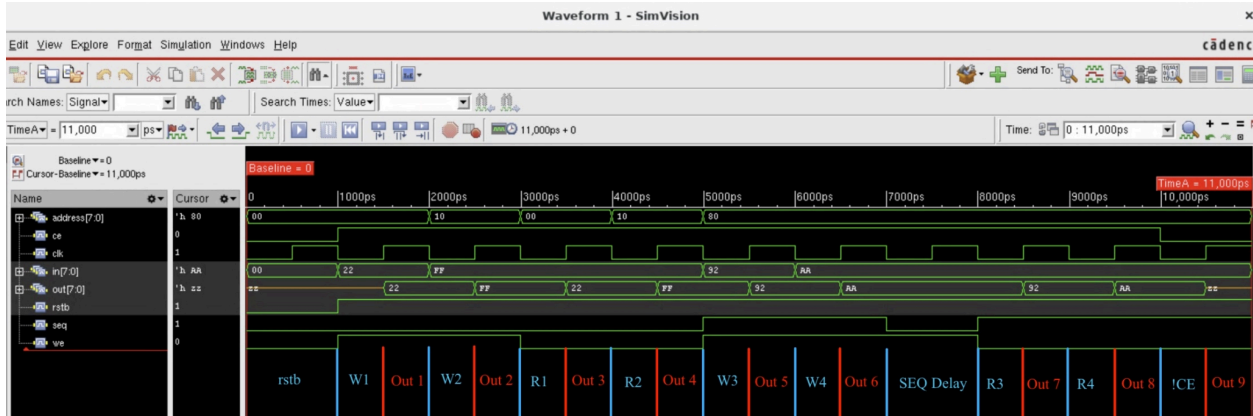


Figure 5: Results of Xcelium simulation for Memory Module with labelled Test Cases (inputs on clock negative edge) and Outputs (outputs on clock positive edge)

In this diagram, on each blue line, the inputs are put into the FSM on the falling edge of the clock and the results can be observed on the red line (rising edge of clock). Each test is named by its operation (W = write, R = read), directly followed by Out # corresponding to each test. Expected and actual results are shown in Table 6.

Table 6: Validation Table for Memory module

Test Case	Inputs	Expected Result	Actual Result
rstb	rstb = 0	Mem clear	—
W1	address = 8'h00 ce = 1 in = 8'h22 seq = 0 we = 1	Write to Mem Out 1 = 8'h22	Out 1 = 8'h22
W2	address = 8'h10 ce = 1 in = 8'hff seq = 0 we = 1	Write to Mem Out 2 = 8'hff	Out 2 = 8'hff

R1	address = 8'h00 ce = 1 in = 8'hff seq = 0 we = 0	Read from 8'h00 Out 3 = 8'h22	Out 3 = 8'h22
R2	address = 8'h10 ce = 1 in = 8'hff seq = 0 we = 0	Read from 8'h10 Out 4 = 8'hff	Out 4 = 8'hff
W3	address = 8'h80 ce = 1 in = 8'h92 seq = 1 we = 1	Write to Mem + 0 Out 5 = 8'h92	Out 5 = 8'h92
W4	address = 8'h80 ce = 1 in = 8'haa seq = 1 we = 1	Write to Mem + 1 Out 6 = 8'haa	Out 6 = 8'haa
SEQ delay	seq = 0	Allocate one clock cycle for SEQ to reset	–
R3	address = 8'h80 ce = 1 in = 8'haa seq = 1 we = 0	Read from 8'h80 + 0 Out 7 = 8'h92	Out 7 = 8'h92
R4	address = 8'h80 ce = 1 in = 8'haa seq = 1 we = 0	Read from 8'h80 + 1 Out 8 = 8'haa	Out 8 = 8'haa
!CE	ce = 0	Turn off chip enable Out 9 = 8'bz	Out 9 = 8'bz

5. SDC file, Timing report and Area report

SDC file:

```
create_clock -name clk -period 1 -waveform { 0 0.5 } [get_ports clk]
```

```
# ----- Input constraints -----
```

```
set_input_delay -clock clk -max 0.4 [get_ports {rstb in ce we seq address}]  
set_input_delay -clock clk -min -0.4 [get_ports {rstb in ce we seq address}]
```

```
# ----- Output constraints -----
```

```
set_output_delay -clock clk -max 0.4 [get_ports out]  
set_output_delay -clock clk -min -0.4 [get_ports out]
```

```
set_max_delay 0.4 -from [all_inputs] -to [all_outputs]
```

```
# Assume 50fF load capacitances everywhere:
```

```
set_load 0.050 [all_outputs]
```

```
# Set 10fF maximum capacitance on all inputs
```

```
set_max_capacitance 0.010 [all_inputs]
```

```
# set clock uncertainty of the system clock (skew and jitter)
```

```
#set_clock_uncertainty -setup 0.03 [get_clocks clk*]
```

```
#set_clock_uncertainty -hold 0.06 [get_clocks clk*]
```

```
# set maximum transition at output ports
```

```
set_max_transition 0.07 [current_design]
```

```
# set_attr use_scan_seqs_for_non_dft false
```

Table 8: Screenshots of area, cell counts and timing reports from synthesis

Report type	Screenshot
Area/Cell Counts	<pre> Instance Module Cell Count Cell Area Net Area Total Area ----- memory 4428 16757.202 7362.864 24120.066 </pre>
Timing	<pre> Path 1: MET (0 ps) Setup Check with Pin out_reg[1]/CK->D Group: clk Startpoint: (F) address[0] Clock: (R) clk Endpoint: (F) out_reg[1]/D Clock: (R) clk Capture Launch Clock Edge:+ 1000 0 Drv Adjust:+ 0 0 Src Latency:+ 0 0 Net Latency:+ 0 (I) 0 (I) Arrival:= 1000 0 Setup:- 41 Required Time:= 959 Launch Clock:- 0 Input Delay:- 400 Data Path:- 559 Slack:= 0 Exceptions/Constraints: input_delay 400 memory.sdc_line_7_19_1 #----- # Timing Point Flags Arc Edge Cell Fanout Load Trans Delay Arrival Instance # (fF) (ps) (ps) (ps) Location #----- address[0] - - F (arrival) 2 4.9 0 0 400 (-,-) g167214/ZN - A->ZN R INV_X1 2 5.4 15 18 418 (-,-) g167154/ZN - A1->ZN F NOR2_X2 1 6.1 8 12 430 (-,-) g167146/ZN - A->ZN R AOI21_X4 3 7.1 22 39 469 (-,-) g167037/ZN - A->ZN F INV_X1 1 3.2 8 13 482 (-,-) g166242/ZN - A1->ZN R NAND2_X2 1 6.3 13 18 500 (-,-) g166228/ZN - A1->ZN F NAND2_X4 6 23.2 14 23 523 (-,-) g165678/ZN - A->ZN R INV_X2 5 12.5 17 27 549 (-,-) g165471/ZN - A1->ZN F NAND2_X2 6 11.5 14 24 574 (-,-) g165430/ZN - A->ZN R INV_X2 8 24.0 30 40 614 (-,-) g165443/ZN - A->ZN F INV_X4 32 53.3 18 31 646 (-,-) g165074/ZN - A->ZN R OAI21_X1 1 2.0 20 26 671 (-,-) g164895/ZN - A2->ZN F AOI22_X1 1 1.8 20 18 689 (-,-) g163969_6083/ZN - B2->ZN R OAI221_X1 1 2.0 37 56 746 (-,-) g163874_4547/ZN - A4->ZN F NAND4_X1 1 1.8 17 34 780 (-,-) g163830_1840/ZN - A->ZN R AOI221_X1 1 1.9 43 75 855 (-,-) g161398_8757/ZN - C2->ZN F OAI221_X1 1 1.8 20 38 893 (-,-) g161393_2703/ZN - A2->ZN R NOR3_X1 1 1.9 28 51 944 (-,-) g161385_7118/ZN - A1->ZN F NAND2_X1 1 1.4 10 16 959 (-,-) out_reg[1]/D <<< - F DFF_X1 1 - - 0 959 (-,-) #----- </pre>

Appendix

1. Problem 1: RTL code – transition_FSM.v

```
`timescale 1ns/10ps

module transition_FSM(clk, rstb, in, out);
    input clk, rstb, in;
    output out;
    reg [1:0] Sreg;
    parameter [1:0] S00 = 2'b00,
        S01 = 2'b01,
        S10 = 2'b10,
        S11 = 2'b11;

    // Output logic
    assign out = (~Sreg[1] && Sreg[0]) || (Sreg[1] && ~Sreg[0]);

    // Next state and State Memory
    always @(posedge clk or negedge rstb)
        if (!rstb) Sreg <= S00; // Reset on negative rstb
        else begin
            case(Sreg)
                S00: if (in == 0) Sreg <= S00;
                     else Sreg <= S01;
                S01: if (in == 0) Sreg <= S10;
                     else Sreg <= S11;
                S10: if (in == 0) Sreg <= S00;
                     else Sreg <= S01;
                S11: if (in == 0) Sreg <= S10;
                     else Sreg <= S11;
                default Sreg <= S00; // Default state for Sreg
            endcase
        end
endmodule
```

2. Problem 1: TB code – transition_FSM_test.v

```
`timescale 1ns/10ps

module transition_FSM_test2;
    reg clk, rstb, in;
    wire out;

    // Initialize FSM
    transition_FSM test_FSM2(.clk(clk), .rstb(rstb), .in(in), .out(out));

    // Initialize simulated clock with 0.5ns period
    initial begin
        clk <= 0;
    end

    always begin
        #0.25 clk = ~clk;
    end

    initial begin
        in <= 1'b0;
        rstb <= 1'b0;
        #0.5
        rstb <= 1'b1;

        // Test 1: 00 -> 00
        in <= 1'b0;
        #0.5

        // Test 2: 00 -> 01
        in <= 1'b1;
        #0.5

        // Test 3: 01 -> 10
        in <= 1'b0;
        #0.5

        // Test 4: 10 -> 01
        in <= 1'b1;
```

```

#0.5

// Test 5: 01 -> 11
in <= 1'b1;
#0.5

// Test 6: 11 -> 11
in <= 1'b1;
#0.5

// Test 7: 11 -> 10
in <= 1'b0;
#0.5

// Test 8: 10 -> 00
in <= 1'b0;
#0.5
$finish;
end
endmodule

```

3. Problem 2: RTL code – memory.v

```

`timescale 1ns/10ps

module memory(clk, we, rstb, address, ce, in, out, seq);
    input clk, we, rstb, ce, seq;
    input [7:0] address, in;
    output reg [7:0] out;

    // Creating seq memory
    reg [7:0] seq_count;
    // Creating memory
    reg [7:0] mem [0:255];
    integer i;

    always @(posedge clk or negedge rstb) begin
        if(!rstb) begin // 1. Reset all memory on !rstb
            for (i = 0; i < 256; i = i + 1) begin
                mem[i] <= 8'b00000000;
            end
        end
    end
endmodule

```

```

    seq_count <= 8'b00000000;
    out <= 8'bz;
end else begin
    if (!ce) begin          // 2. Output is high impedance if ce == 0
        out <= 8'bz;
    end else begin
        if (seq) begin      // 3. Update seq
            seq_count <= seq_count + 1;
        end else begin
            seq_count <= 8'b00000000;
        end

        // Note: Since all statements are non-blocking, when seq_count increments, the
memory access
        // does not use that value until the subsequent clock cycle

        if (we) begin
            mem[address + seq_count] <= in; // 4. Write to memory
            out <= in;
        end else begin
            out <= mem[address + seq_count]; // 5. Read from memory
        end
    end
end
end
end
endmodule

```

4. Problem 2: TB code – memory_test.v

```
`timescale 1ns/10ps

module memory_test;
    reg clk, we, rstb, ce, seq;
    reg [7:0] address, in;
    wire [7:0] out;

    memory test_memory(.clk(clk), .we(we), .rstb(rstb), .ce(ce), .seq(seq), .address(address),
.in(in), .out(out));

    // Initialize simulated clock with 1ns period
    initial begin
        clk <= 0;
    end

    always begin
        #0.5 clk = ~clk;
    end

    // Inputs change approximately on falling edge of clk
    initial begin
        // Reset memory
        we <= 0;
        ce <= 0;
        seq <= 0;
        address <= 8'b00000000;
        in <= 8'b00000000;
        rstb <= 0;
        #1
        rstb <= 1;

        // Section 1: SEQ = 0
        ce <= 1'b1;
        seq <= 1'b0;

        // Write #1
        we <= 1'b1;
        address <= 8'b00000000;
        in <= 8'b00100010;
```

#1

// Write #2

address <= 8'b00010000;

in <= 8'b11111111;

#1

// Read #1

we <= 1'b0;

address <= 8'b00000000;

#1

// Read #2

address <= 8'b00010000;

#1

// Section 2: SEQ = 1

ce <= 1'b1;

seq <= 1'b1;

// Write #1

we <= 1'b1;

address <= 8'b10000000;

in <= 8'b10010010;

#1

// Write #2 (This should now write to 8'b10000001)

in <= 8'b10101010;

#1

// Now set SEQ to 0 for one cycle

seq <= 1'b0;

#1

// Read #1

seq <= 1'b1;

we <= 1'b0;

address <= 8'b10000000;

#1

```
// Read #2 (This should now read from 8'b10000001)
#1

// Turning off memory (ce = 0)
ce <= 0;
#1 $finish;
end
endmodule
```